

Week11

Task1:

Develop a multithreaded counter application using POSIX threads (pthread_create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_THREADS 5
#define ITERATIONS 100000
int counter = 0;
void *increment_counter(void *arg){
    int i;
    for (i = 0; i < ITERATIONS; i++){
        counter++;
    }
    return NULL;
}
int main(){
    pthread_t threads[NUM_THREADS];
    int i;
    printf("Initial counter: %d\n", counter);
    for (i = 0; i < NUM_THREADS; i++){
        if (pthread_create(&threads[i], NULL, increment_counter, NULL) != 0){
            perror("pthread_create");
            return 1;
        }
    }
    for (i = 0; i < NUM_THREADS; i++){
        pthread_join(threads[i], NULL);
    }
    printf("Expected counter: %d\n", NUM_THREADS * ITERATIONS);
    printf("Actual counter: %d\n", counter);
    if (counter != NUM_THREADS * ITERATIONS)
        printf("Race condition occurred!\n");
    else
        printf("No race condition observed.\n");
    return 0;
}
```

Output:

```
root@DESKTOP-VMN932K:~# ./posix2
Initial counter: 0
Expected counter: 500000
Actual counter: 402525
Race condition occurred!
root@DESKTOP-VMN932K:~#
```

Task2:

Modify the previous program using mutex locks (pthread_mutex_lock, pthread_mutex_unlock) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#define NUM_THREADS 5
#define ITERATIONS 100000
int counter = 0;
pthread_mutex_t lock;
void *increment_counter(void *arg){
    int i;
    for (i = 0; i < ITERATIONS; i++){
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return NULL; }
int main(){
    pthread_t threads[NUM_THREADS];
    int i;
    pthread_mutex_init(&lock, NULL);
    printf("Initial counter: %d\n", counter);
    for (i = 0; i < NUM_THREADS; i++){
        if (pthread_create(&threads[i], NULL,
                        increment_counter, NULL) != 0){
            perror("pthread_create");
            return 1;
        }
    }
    for (i = 0; i < NUM_THREADS; i++){
        pthread_join(threads[i], NULL);
    }
    printf("Expected counter: %d\n",
        NUM_THREADS * ITERATIONS);
    printf("Actual counter: %d\n", counter);
    if (counter == NUM_THREADS * ITERATIONS)
        printf("Counter is correct. Race condition prevented.\n");
    else
        printf("Race condition occurred.\n");
    pthread_mutex_destroy(&lock);
    return 0;
}

```

Output:

```

root@DESKTOP-VMN932K:~# ./mutex
Initial counter: 0
Expected counter: 500000
Actual counter: 500000
Counter is correct. Race condition prevented.
root@DESKTOP-VMN932K:~#

```